Laboratório Funcional · Aula 12

Vanessa Beatriz G. Beltrame · RA 24034783-2 · Turma C
Gerado em 16/10/2026, 22:51:45 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

Função agora(): Ela é impura porque retorna a data e a hora atuais, ou seja, o resultado muda conforme o momento em que é executada.
Função registra(msg): Ela é impura porque utiliza print() para exibir uma mensagem na tela, causando um efeito colateral.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: `1: A-1 A-2`

Linha 4: `4: 14`

Linha 7: `7: [7, 3, 9] [3, 7, 9]`

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

```
1: A-1 A-2
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Por que devolve valores diferentes: Porque a função é impura e depende de um estado global mutável (contador), que é alterado a cada chamada. Por que dificulta os testes: Porque perde a transparência referencial, o que obriga a preparar e limpar o contexto (o estado) antes de cada teste e pode causar falhas consoante a ordem de execução.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

g(4) = 3x4 = 12 f(12)=12+2=14
f(4)=4+2=6 g(6)=3x6=18

A composição de funções não é comutativa. A ordem de aplicação altera o resultado porque: Em compor(f, g), você primeiro multiplica por 3 e depois soma 2. Em compor(g, f), você primeiro soma 2 e depois multiplica tudo por 3.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria e retorna uma lista nova ordenada, deixando a lista original intacta. notas.sort() modifica a lista original diretamente no lugar (in-place). sorted(notas) segue o estilo funcional, pois respeita a imutabilidade (evita alterar estados existentes e não gera efeitos colaterais).

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

primeiro elemento da lista

Previsão: (cdr '(A B C))

retorna o resto da lista

Previsão: (cons '(A B) '(C D))

insere o primeiro argumento como o primeiro elemento da segunda lista

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```

```
22
14
25
8
negativo
Exception in area: application: not a procedure;
 expected a procedure that can be applied to arguments
  given: #<void>
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

Ele serve para indicar que a lista seguinte é um dado (uma coleção de símbolos) e não uma expressão a ser executada ou avaliada. Sem o apóstrofo, o Scheme tentou avaliar (A B C) como se fosse uma chamada de função, tratando o A como o operador (a função) e B e C como os argumentos passados para ele.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
(define (mostra x) (display x) (newline))

(mostra (car (cdr (cdr '(A B C)))))

Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Vai funcionar perfeitamente. O Scheme exige por padrão a otimização de chamada em cauda (tail-call optimization), fazendo com que o interpretador reaproveite o registro de ativação e execute um milhão de chamadas em espaço constante, como um laço. Em Python não funcionaria (geraria um erro de limite de recursão excedido). O Python padrão não otimiza chamadas em cauda e impõe um limite rígido ao tamanho da pilha de execução.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "main.py", line 8, in <module>
```

```
    print(conta_ate(1_000_000, 0))
  File "main.py", line 5, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
  File "main.py", line 5, in conta_ate
    print(conta_ate(1_000_000, 0))
  [Previous line goes on repetidamente repetidas vezes...]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

A operação (ex: multiplicar) não pode ser concluída imediatamente, pois precisa esperar o retorno da próxima chamada. Isso faz com que as chamadas se acumulem na pilha de memória. Como a chamada recursiva é a última instrução e o valor acumulado é passado diretamente por parâmetro, nada fica pendente. O interpretador/compilador reaproveita o mesmo registro de ativação, mantendo o uso de memória constante (como um laço)

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

Scheme: Exige e implementa por padrão a otimização de chamada em cauda, o que faz o interpretador reaproveitar o mesmo registro de ativação da pilha e executar as chamadas em espaço constante, como se fosse um laço. Python: Não realiza essa otimização, acumulando um novo registro de ativação na pilha de execução a cada chamada até atingir o limite máximo permitido e lançar o RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3   ok
soma-quadrados: 14   ok
pares: (2 4 6)   ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
; 1) Quantos elementos a lista tem?
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

; 2) Soma dos quadrados dos elementos
(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

; 3) Retorna apenas os números pares da lista
(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

Parte mais difícil: Compreender a lógica de construir novos resultados combinando chamadas recursivas sem utilizar variáveis mutáveis.
null?: Utilizada em todas as funções para verificar se a lista chegou ao fim (o caso base).
car: Pega o primeiro elemento da lista. Foi usada em soma-quadrados (para calcular o quadrado do número) e em pares (para testar se é par e incluí-lo na nova lista). Não foi necessária em tamanho.
cdr: Utilizada em todas as funções para avançar para o restante da lista na próxima chamada recursiva.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares: Imprime os primeiros 5 elementos da lista infinita de quadrados: [0, 1, 4, 9, 16].
primeiro 42 (div 1 0): Imprime 42.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
Timeout limit exceeded (10s)
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa avaliação preguiçosa (lazy evaluation). Ele só calcula os dados sob demanda e apenas na quantidade necessária (por exemplo, como o take 5 pede só 5 itens, o programa gera apenas esses cinco e para). A função primeiro descarta o segundo argumento. Como o Haskell só avalia uma expressão quando ela é realmente usada, a divisão por zero nunca chega a ser executada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

A função percorreu os quadrados (0, 1, 4, 9, 16) até encontrar o 16. Como o operador || (ou) é avaliado de forma preguiçosa e encontrou um True, a busca parou imediatamente e retornou sucesso. A função padrão member assume que qualquer lista é desordenada. Como o 17 não é um quadrado perfeito, a função não tem como adivinhar se ele aparecerá mais à frente, então continua gerando e testando os próximos números infinitamente (25, 36, 49, 64...), resultando no estouro de tempo (Timeout). A solução (conforme discutido no livro na p. 670) consiste em adicionar uma condição (guard) para verificar se o elemento atual é maior do que o buscado.

```
memberOrd :: Int -> [Int] -> Bool
memberOrd b [] = False
```

```
memberOrd b (a:x)
 | a == b   = True
 | a > b    = False
 | otherwise = memberOrd b x
```

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

console.log(mais10(5)); imprime 15. console.log([1, 2, 3].map(vezes(5))); imprime [5, 10, 15].

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Ela recebe o argumento n = 10 e cria a função interna (x) => x + n.
Em linguagens sem suporte a fechamentos, a variável local n morreria assim que o escopo de somador(10) terminasse.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Devido a ordem em que as funções são aplicadas:
f.compose(g).apply(4) aplica g primeiro e depois f:
g(4)=3X4=12 F= 12+2+14

f.andThen(g).apply(4) aplica o f primeiro e depois o g:
f(4)=4+2=6 g=3x6=18

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
```

```
            .sum();

        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
precos = [50.0, 120.0, 80.0, 200.0]

com_desconto = [p * 0.9 if p > 100 else p for p in precos]

print(com_desconto)
```

Saídas das suas versões

```
244

[50.0, 108.0, 80.0, 180.0]
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

versão funcional (pipelines de stream ou compreensões de lista) costuma ser considerada mais fácil de ler, pois ela é descritiva. O laço tradicional (for) é consideravelmente mais fácil de depurar.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil porque é declarativa: ela descreve diretamente o que fazer — "gera a lista, eleva ao cubo e soma" —, eliminando o controle manual de contadores e variáveis mutáveis do for em C. Aceitaria (Foco na clareza e manutenção):
Sistemas Web e corporativos: O gargalo é o banco de dados e a rede, não a CPU.

Análise de dados e protótipos: Onde evitar bugs e programar mais rápido importa mais do que milissegundos.

NÃO aceitaria (Foco na velocidade extrema):

Sistemas embarcados e de tempo real: Carros, aviação e equipamentos médicos.

Jogos e motores gráficos: Exigem taxa de quadros alta e controle absoluto de memória.

Mercado financeiro de alta frequência: Onde microssegundos fazem diferença.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Tive um pouco mais de dificuldade em entender algumas perguntas e de responder elas.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

não respondido